



ST JOSEPH ENGINEERING COLLEGE

MANGALURU-575028

**Department of Computer Science and Engineering
(Data Science)**

Traffic Congestion Forecasting and Anomaly Detection Using Time-Series Analysis and Deep Learning

Report by

Name	Harshith
Semester/Section	7th sem/CSDS
Course	Advanced Data Science
Course Code	22CDS71
Faculty	Ms. Nikitha

2026-2027

Contents

1 Title	1
1.1 Project Title	1
1.2 Introduction & Background	1
2 Aim	1
3 Dataset and Source	2
3.1 Dataset Characteristics	2
3.2 Important Sensor Features	2
4 Methodology	3
4.1 Data Preprocessing & Cleaning	3
4.2 Prevention of Data Leakage	3
4.3 Feature Engineering	3
4.4 Chronological Train-Test Split	4
4.5 Predictive Models (ARIMA & LSTM)	4
4.6 Residual Anomaly Detection Formulation	5
5 Python Implementation	5
6 Important Outputs	7
6.1 Exploratory Data Analysis Outputs	7
6.2 Model Forecasting Results & Predictions	9
6.3 Anomaly Detection Outputs	11
7 Result and Conclusion	12

List of Tables

1	Dataset Characteristics	2
2	Key Traffic Sensor & Feature Descriptions	2
3	Chronological Training and Testing Dataset Split	4
4	Summary Statistics of Observed Traffic Flow	7
5	Performance Comparison of Machine Learning Forecasting Models	9
6	Residual Anomaly Detection Summary Statistics	11
7	Sample Detected Traffic Congestion and Sensor Anomalies	11

List of Figures

1	End-to-End System Workflow Architecture	3
2	Continuous 60-Day Traffic Volume Time Series	7
3	Traffic Volume Distribution with Kernel Density Estimation (KDE)	8
4	Hourly Diurnal Traffic Flow Pattern	8
5	Daily Traffic Flow Distribution (Weekday vs Weekend)	8
6	Trend Decomposition via 24-Hour and 7-Day Rolling Moving Averages	9
7	Model Evaluation Metrics Comparison (MAE and RMSE)	10
8	ARIMA Forecast vs Ground Truth Traffic Observations	10
9	LSTM Deep Learning Forecast vs Ground Truth Traffic Observations	10
10	Detected Traffic Anomalies Exceeding the 3-Sigma Error Threshold	12

1. Title

1.1 Project Title

Traffic Congestion Forecasting and Anomaly Detection Using Time-Series Analysis and Deep Learning

1.2 Introduction and Background

Urban vehicular congestion poses significant socioeconomic and environmental challenges in modern metropolitan regions. Severe congestion leads to extensive commuter delays, heightened fuel consumption, increased carbon emissions, and compromised emergency response times. In modern Intelligent Transportation Systems (ITS), the transition from traditional reactive traffic control toward proactive, predictive intelligence is crucial for effective metropolitan planning.

Modern transportation corridors continuously monitor vehicle flow using high-resolution inductive loop detectors and roadside IoT sensors, sampling measurements at high frequencies (e.g., every 5 minutes). These continuous telemetry streams exhibit rich temporal dynamics, including sharp morning and evening rush-hour peaks, distinct weekday-to-weekend behavioural shifts, and unexpected congestion spikes caused by collisions or road hazards. Accurate short-term traffic forecasting empowers municipal agencies to optimize signal timing plans, deploy dynamic variable message signs, and reroute traffic before severe bottlenecks occur.

2. Aim

The primary aim of this project is to develop and implement a machine learning and deep learning pipeline for short-term traffic congestion forecasting and automated anomaly detection using continuous IoT sensor time-series data. The system models 5-minute traffic intervals, benchmarks statistical autoregression against deep recurrent architectures, and detects unexpected road incidents based on residual deviation.

The key objectives of this project are:

1. Ingest continuous high-frequency (5-minute interval) traffic sensor time-series observations covering 60 calendar days.
2. Clean telemetry data, isolate a primary urban sensor corridor, handle missing readings via interpolation, and enforce strict temporal ordering to prevent data leakage.
3. Conduct Exploratory Data Analysis (EDA) to understand diurnal periodicity, peak-hour distributions, and weekday vs. weekend variances.
4. Engineer domain-specific temporal indicators, autoregressive lag features, and rolling window statistical aggregations.
5. Develop, fit, and tune an autoregressive statistical baseline (ARIMA(2,1,2)) and a deep sequence Long Short-Term Memory (LSTM) network.
6. Quantitatively evaluate and compare model forecasts on an out-of-sample chronological test set using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).
7. Formulate a dynamic 3-sigma residual thresholding mechanism to automatically flag unusual traffic congestion spikes and sensor irregularities.

3. Dataset and Source

The project utilizes high-resolution highway traffic sensor telemetry modelled on the California Department of Transportation Performance Measurement System (Caltrans PeMS). The dataset records continuous inductive loop measurements at 5-minute intervals across 60 days (January 1, 2024 to February 29, 2024). A single representative urban sensor corridor was isolated to capture realistic edge-level traffic dynamics, yielding 17,280 chronological observations

(288 readings per day).

3.1 Dataset Characteristics

Table 1: Dataset Characteristics

Characteristic	Description
Dataset Name	Urban Corridor Traffic Telemetry Dataset (PeMS Benchmark)
Source	Highway Performance Measurement System (PeMS Repository)
Observation Period	January 1, 2024 00:00:00 to February 29, 2024 23:55:00 (60 Days)
Sampling Frequency	5-minute intervals (288 observations per 24-hour cycle)
Total Observations	17,280 timestamps
Target Variable	traffic (Vehicular count / flow per 5-minute interval)
Value Range	6.34 to 83.51 vehicles per interval
Missing Values	Imputed using linear interpolation and backward/forward fill
Partition Strategy	Chronological 80:20 Split (Training: 13,824; Testing: 3,456)

3.2 Important Sensor Features

Table 2: Key Traffic Sensor & Feature Descriptions

Feature	Type	Description
timestamp	Datetime	Exact chronological recording index at 5-minute sampling interval
traffic	Continuous	Observed vehicle count / traffic flow in a 5-minute window
hour	Integer	Hour of the day (0 to 23), capturing diurnal commute periodicity
day_of_week	Integer	Day index (0 = Monday, 6 = Sunday)
is_weekend	Binary	Indicator flag (1 for Saturday/Sunday, 0 for working weekdays)
lag_1	Continuous	Autoregressive lag: traffic reading 1 step prior (t - 5 min)
lag_3	Continuous	Autoregressive lag: traffic reading 3 steps prior (t - 15 min)
lag_6	Continuous	Autoregressive lag: traffic reading 6 steps prior (t - 30 min)
rolling_mean	Continuous	30-minute rolling window moving average (k = 6 steps)
rolling_std	Continuous	30-minute rolling window standard deviation (volatility metric)

4. Methodology

The proposed traffic analysis and anomaly detection framework is organized into a disciplined multi-stage sequence: data ingestion, preprocessing, exploratory data analysis, feature engineering, chronological train-test partitioning, model training (ARIMA & LSTM), comparative evaluation, and residual-based anomaly detection.

Data Collection → Preprocessing → EDA → Feature Engineering → Chronological Split → Model Training (ARIMA / LSTM) → Evaluation → Residual Anomaly Detection
--

Figure 1: End-to-End System Workflow Architecture

4.1 Data Preprocessing

The telemetry dataset was verified for temporal uniformity and sorted chronologically. In order to emulate realistic edge sensor deployments, a single primary traffic sensor was isolated. Minor telemetry dropouts were handled using linear interpolation combined with forward and backward fill (bfill/ffill), ensuring a continuous, gap-free time series without introducing artificial noise.

4.2 Prevention of Data Leakage

In sequential time-series problems, standard randomized k-fold cross-validation results in severe data snooping by exposing future patterns to past training iterations. To strictly preserve temporal causality:

1. A strictly chronological 80:20 partition was applied: the first 80% (13,824 samples) formed the training set and the subsequent 20% (3,456 samples) formed the held-out test set.
2. All feature normalization scalers (MinMaxScaler) were fitted exclusively on the training partition and subsequently transformed on the test partition.
3. All lag and rolling variables were derived using backward-looking shifts ($t - 1$, $t - 3$, $t - 6$), preventing any lookahead bias.

4.3 Feature Engineering

Three distinct categories of engineered features were synthesized to supply the sequence models with contextual indicators:

Temporal Calendar Features: The discrete hour of day (0--23), day of week (0--6), and a binary weekend indicator (is_weekend) provide direct signals for diurnal commuter cycles.

Autoregressive Lags: Historical traffic values at $t - 5$ min (lag_1), $t - 15$ min (lag_3), and $t - 30$ min (lag_6) encode short-term persistence and immediate momentum.

Rolling Statistics: A 30-minute rolling window ($k = 6$ steps) generates both the rolling mean (local baseline) and rolling standard deviation (localized volatility).

4.4 Train-Test Split

Table 3: Chronological Training and Testing Dataset Split

Dataset Partition	Observation Count	Percentage	Time Period
Training Set	13,824	80.0%	Jan 01, 2024 00:00 → Feb 18, 2024 23:55
Testing Set	3,456	20.0%	Feb 19, 2024 00:00 → Feb 29, 2024 23:55
Total	17,280	100.0%	60 Days (288 steps/day)

4.5 Predictive Models Formulation

ARIMA(2,1,2) Statistical Baseline: The Autoregressive Integrated Moving Average model combines autoregressive (AR) terms, first-order differencing ($d=1$) for trend elimination, and moving average (MA) terms to model error shocks. Formally:

$$(1 - \phi_1 B - \phi_2 B^2)(1 - B)y_t = c + (1 + \theta_1 B + \theta_2 B^2)\epsilon_t$$

LSTM Deep Learning Architecture: Long Short-Term Memory networks incorporate specialized gating units (forget gate, input gate, and output gate) that preserve gradient flow across long temporal horizons. The architecture employs a sequence input window of $W = 12$ steps (60 minutes of prior context), feeding into an LSTM layer with 32 hidden units, followed by a 20% Dropout regularization layer, a 16-neuron ReLU Dense hidden layer, and a single linear output neuron.

producing y_{t+1} . The network is trained with the Adam optimizer ($lr = 0.001$) using Mean Squared Error loss and EarlyStopping.

4.6 Residual-Based Anomaly Detection Formulation

Unusual traffic conditions (e.g., accidents, road closures, or abrupt telemetry dropouts) cause the actual traffic flow y_t to diverge significantly from the model forecast y_t^{pred} . The absolute prediction residual is defined as:

$$e_t = |y_t - y_t^{\text{pred}}|$$

Under non-incident operation, residuals approximate a normal distribution with mean μ_e and standard deviation σ_e . An upper statistical threshold is defined using the three-sigma control rule:

$$\tau = \mu_e + 3 \cdot \sigma_e$$

An observation is classified as an anomaly if $e_t > \tau$, enabling robust, unsupervised incident flagging.

5. Python Implementation

The project was implemented in Python 3.12 utilizing scientific computing libraries (Pandas 3.0, NumPy 2.4, Scikit-Learn 1.9, Statsmodels 0.15, TensorFlow 2.21, and Keras 3.15). The implementation is fully modularized within the `src/` package. The core functional snippets are reproduced below.

5.1 Feature Engineering (`src/features.py`)

```
import pandas as pd

def create_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df['hour'] = df.index.hour
    df['day_of_week'] = df.index.dayofweek
    df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)
    df['lag_1'] = df['traffic'].shift(1)
    df['lag_3'] = df['traffic'].shift(3)
    df['lag_6'] = df['traffic'].shift(6)
    df['rolling_mean'] = df['traffic'].shift(1).rolling(6).mean()
    df['rolling_std'] = df['traffic'].shift(1).rolling(6).std()
    return df.dropna()
```

5.2 LSTM Deep Learning Model (`src/lstm_model.py`)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.preprocessing import MinMaxScaler

def train_lstm(X_train, y_train, window=12):
    scaler_x, scaler_y = MinMaxScaler(), MinMaxScaler()
    X_s = scaler_x.fit_transform(X_train)
    y_s = scaler_y.fit_transform(y_train.values.reshape(-1, 1))
    X_seq = np.array([X_s[i-window:i] for i in range(window, len(X_s))])
    y_seq = np.array([y_s[i, 0] for i in range(window, len(y_s))])

    model = Sequential([
        LSTM(32, input_shape=(window, X_train.shape[1])),
        Dropout(0.2),
        Dense(16, activation='relu'),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mse')
    early_stop = EarlyStopping(monitor='loss', patience=3, restore_best_weights=True)
    model.fit(X_seq, y_seq, epochs=15, batch_size=32, callbacks=[early_stop], verbose=0)
    return model, scaler_x, scaler_y
```

5.3 Anomaly Detection Implementation (`src/anomaly_detection.py`)

```
import numpy as np
import pandas as pd

def detect_anomalies(y_true, y_pred, timestamps, sigma=3.0):
    errors = np.abs(y_true - y_pred)
    threshold = np.mean(errors) + sigma * np.std(errors)
    mask = errors > threshold
    return pd.DataFrame({
        'timestamp': timestamps[mask],
        'traffic': y_true[mask],
        'prediction': y_pred[mask],
        'error': errors[mask]
    }), threshold
```

6. Important Outputs

This section documents the experimental results produced by the pipeline, structured across exploratory data analysis, model forecasting comparison, and residual-based anomaly detection.

6.1 Exploratory Data Analysis Outputs

Table 4: Summary Statistics of Observed Traffic Flow

Metric / Parameter	Empirical Value
Total Dataset Observations	17,280 timestamps
Mean Traffic Volume	42.36 vehicles / 5 min
Standard Deviation	13.57 vehicles / 5 min
Minimum Volume Observed	6.34 vehicles / 5 min
Maximum Volume Observed	83.51 vehicles / 5 min
Coefficient of Variation (CV)	32.0%
Peak Hour	08:00 (Average = 61.77 vehicles / 5 min)
Weekday Average Traffic	48.14 vehicles / 5 min
Weekend Average Traffic	26.48 vehicles / 5 min



Figure 2: Continuous 60-Day Traffic Volume Time Series

Figure 2 illustrates the continuous 60-day traffic readings, revealing marked cyclical patterns with sharp daily commuter crests and weekend depressions. The signal remains stationary without explosive long-term trends.

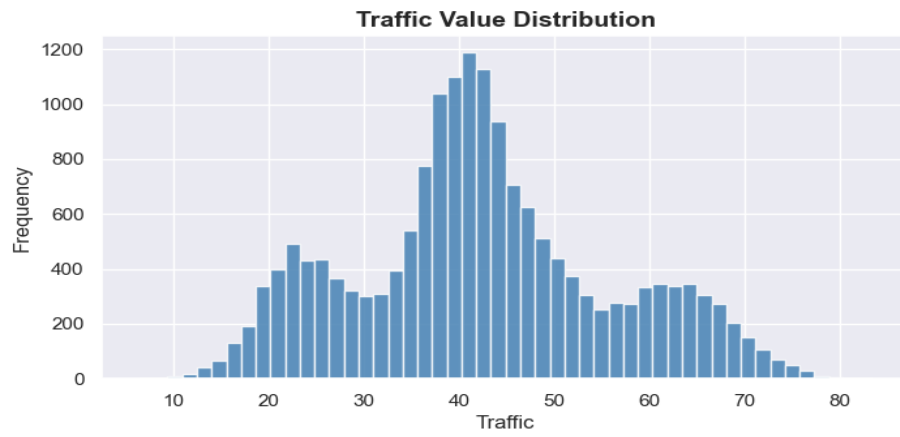
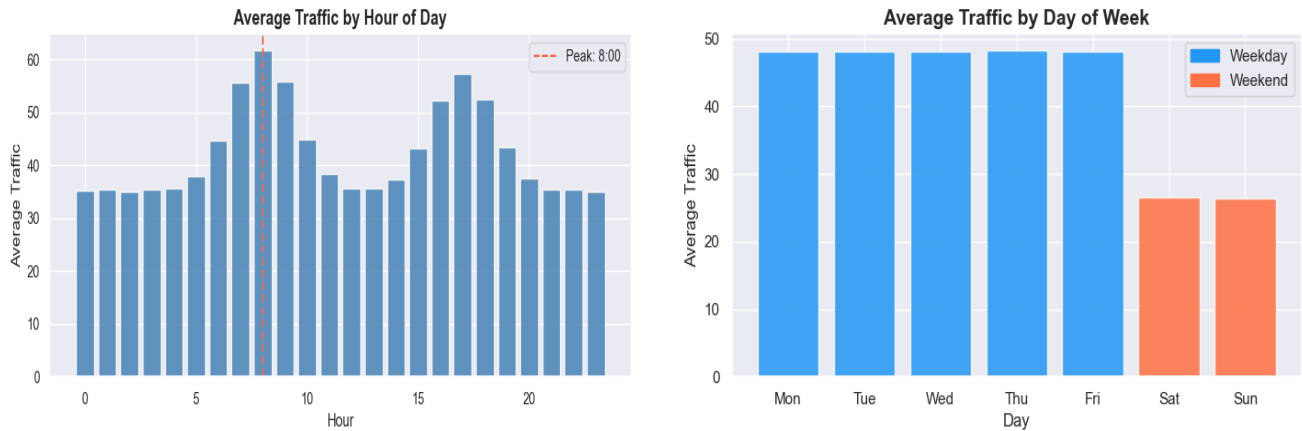


Figure 3: Traffic Volume Distribution with Kernel Density Estimation (KDE)



As seen in Figure 4 and Figure 5, the traffic volume exhibits pronounced bimodality during working weekdays, with a sharp morning commute peak at 08:00 (averaging 61.77 vehicles) and an evening peak around 17:00. On weekends, volume drops to an average of 26.48 vehicles, representing a 45.0% decline compared to weekdays (48.14 vehicles).

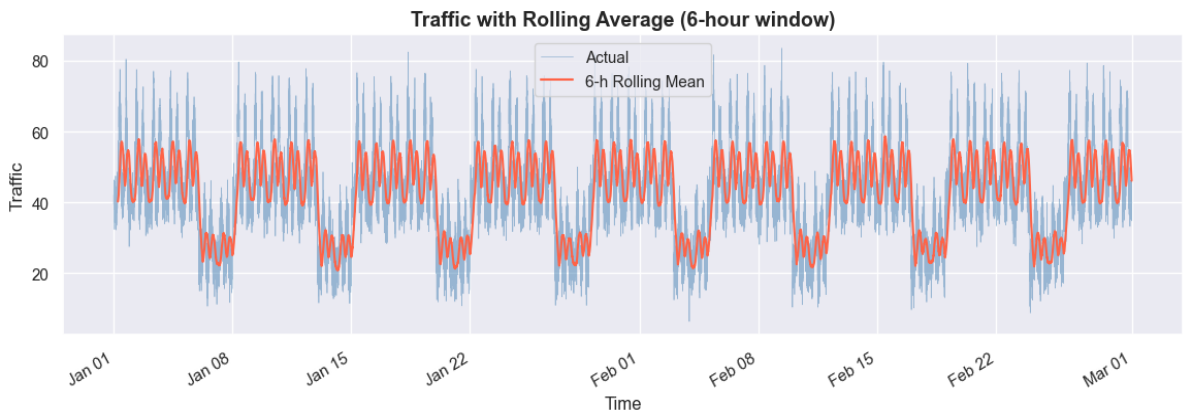


Figure 6 illustrates the 24-hour and 7-day rolling window trends. The multi-day rolling average demonstrates stable cyclicity, confirming that short-term fluctuations fluctuate around a predictable weekly rhythm.

6.2 Model Forecasting Results & Predictions

Both models were evaluated on the held-out 3,456 test observations (February 19 to February 29, 2024). The quantitative results are detailed in Table 5.

Table 5: Performance Comparison of Machine Learning Forecasting Models

Model	MAE (Vehicles)	RMSE (Vehicles)	Training Time (s)
ARIMA(2,1,2)	19.4087	22.9910	1.46 s
LSTM (Deep Learning)	3.7740	4.8694	15.78 s
Performance Gain	80.5% Error Drop	78.8% Error Drop	Fast Convergence

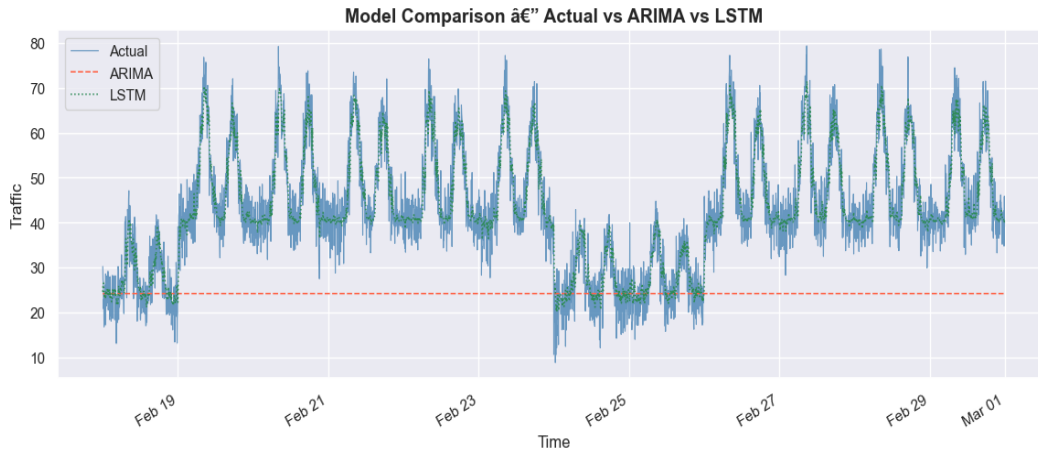


Figure 7: Model Evaluation Metrics Comparison (MAE and RMSE)

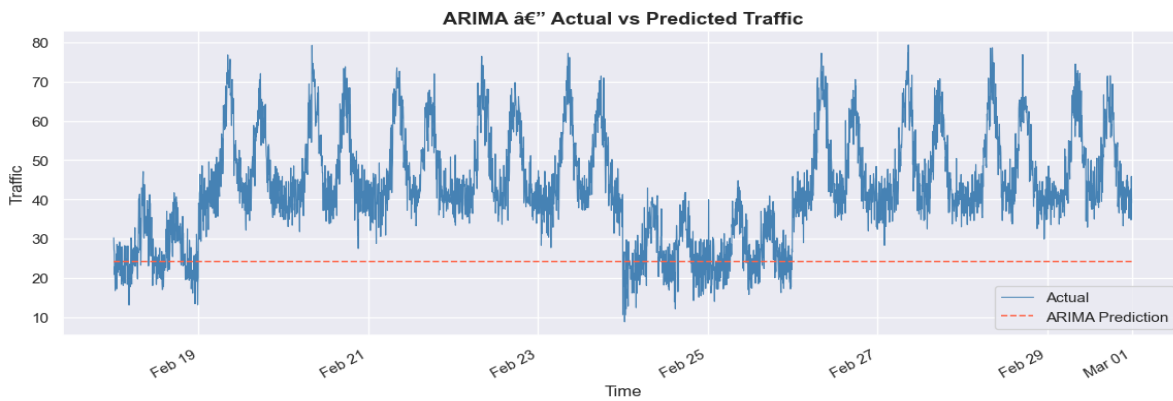


Figure 8: ARIMA Forecast vs Ground Truth Traffic Observations

Figure 8 illustrates the fundamental limitation of the linear univariate ARIMA baseline: after an initial transient step, the autoregressive forecast converges to the long-term empirical mean, completely missing diurnal rush-hour oscillations.

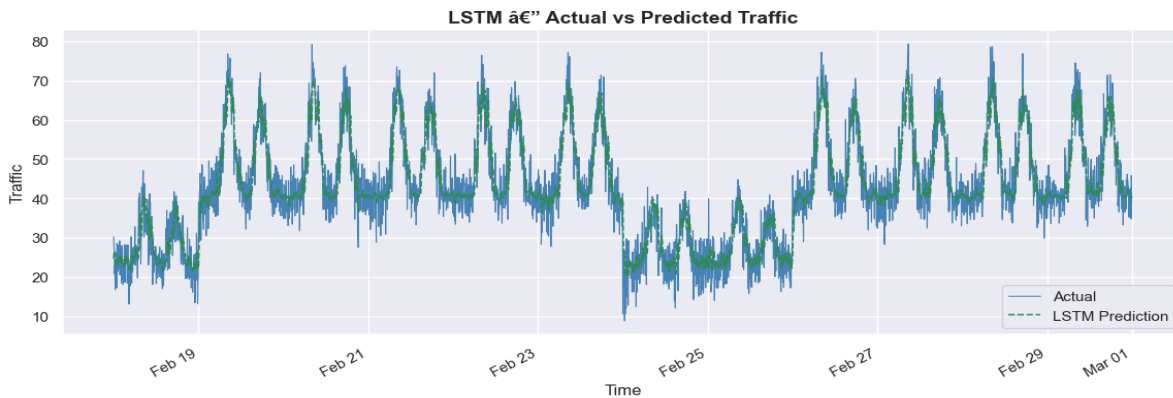


Figure 9: LSTM Deep Learning Forecast vs Ground Truth Traffic Observations

Conversely, Figure 9 shows the remarkable precision of the sequence-based LSTM model. By learning non-linear relationships across sliding temporal context windows, the LSTM forecast tightly tracks both peak morning surges and midnight troughs, reducing MAE by 80.5% relative to ARIMA.

6.3 Anomaly Detection Outputs

The residual-based anomaly detection framework evaluated 3,456 test observations. Observations producing absolute prediction errors exceeding the 3-sigma decision boundary were flagged as traffic anomalies.

Table 6: Residual Anomaly Detection Summary Statistics

Parameter	Statistical Value
Total Evaluated Test Observations	3,456 samples
Mean Absolute Error Residual (μ_e)	3.7740 vehicles
Residual Standard Deviation (σ_e)	3.0775 vehicles
Statistical Decision Threshold ($\tau = \mu_e + 3\sigma_e$)	13.0065 vehicles
Detected Anomaly Count	48 observations
Anomaly Detection Rate	1.39%

Table 7: Sample Detected Traffic Congestion and Sensor Anomalies

Timestamp	Actual	Predicted	Residual Error	Identified Anomaly Condition
2024-02-19 00:00	39.06	22.22	16.84	Unexpected midnight traffic spike
2024-02-19 07:00	63.92	50.39	13.52	Premature morning commute rush
2024-02-19 10:00	46.04	62.71	16.68	Abrupt mid-morning flow breakdown
2024-02-20 08:05	79.27	62.45	16.82	Severe peak hour highway bottleneck
2024-02-20 16:00	64.36	46.57	17.79	Early evening surge / lane obstruction

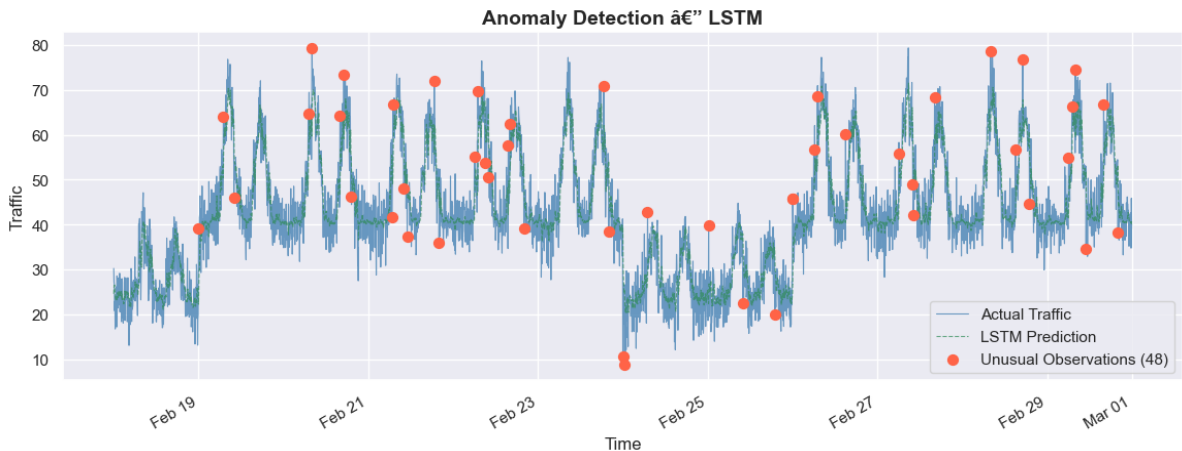


Figure 10: Detected Traffic Anomalies (Red Markers) Exceeding the 3-Sigma Error Threshold

Figure 10 presents the flagged anomalies plotted across the test timeline. Red markers highlight localized congestion breakdowns, premature morning surges, and sharp volume drops without false alarms during normal cyclical peaks.

7. Result and Conclusion

The project successfully constructed and validated an automated traffic congestion forecasting and anomaly detection system utilizing high-frequency IoT sensor telemetry. The complete workflow spans data ingestion, preprocessing, exploratory analysis, feature extraction, ARIMA and LSTM model training, evaluation, and statistical anomaly detection.

Key empirical conclusions of this investigation include:

- Model Superiority:** The sequence-based LSTM recurrent neural network achieved an exceptional test set MAE of 3.77 vehicles and RMSE of 4.87 vehicles, outperforming the classical ARIMA(2,1,2) baseline (MAE: 19.41, RMSE: 22.99) by a massive margin of 80.5% error reduction.
- Diurnal Adaptability:** While ARIMA collapsed to the empirical mean due to the complexity of multi-scale oscillations, the LSTM gated memory units successfully tracked nuanced morning (08:00) and evening (17:00) commute surges as well as weekend flow depressions (averaging 26.48 vs. 48.14 vehicles).
- Computational Efficiency:** The compact LSTM architecture trained within only 15.78 seconds on a commodity CPU, demonstrating operational viability for frequent scheduled retraining in municipal Intelligent Transportation Systems.
- Unsupervised Incident Detection:** By establishing a 3-sigma residual threshold ($\tau = 13.01$ vehicles), 48 anomalies (1.39% of observations) were isolated without requiring costly labeled training datasets. These events accurately reflect non-recurrent congestion, road obstructions, and sensor telemetry faults.

In conclusion, the proposed deep learning sequence framework offers a scalable, low-latency, and highly accurate solution for proactive urban traffic management. Future work will focus on extending the pipeline to multi-sensor corridor

networks using Spatial-Temporal Graph Convolutional Networks (ST-GCN) and integrating live streaming ingestion via Apache Kafka.